

MOVIE TICKET BOOKING SYSTEM

Name-Amanpreet singh

Roll no:08

Section:B2

A. Requirement Analysis

A.1 Functional Requirements

FR1 — List movies. When the customer chooses "List Movies" from the main menu, the system shows every movie currently playing, along with its language and duration. If no movie is currently playing, the system prints a message saying so instead of showing an empty list.

FR2 — List shows for a movie. After the customer selects a movie, the system lists every show scheduled for that movie, showing the screen number and the start time of each show. If the selected movie has no scheduled show, the system reports this clearly rather than leaving the screen blank.

FR3 — Display the seat layout. After the customer selects a show, the system displays every seat belonging to that show, grouped by seat type (SILVER / GOLD / PLATINUM), each marked AVAILABLE or BOOKED. Displaying the layout is a read-only operation and never changes any seat's status.

FR4 — Booking (given). A customer selects one or more seat numbers for a show. If any selected seat is already BOOKED, the whole booking is rejected and no seat changes state. The booking is confirmed only after payment succeeds.

FR5 — Pricing. The price of a seat depends only on its seat type — SILVER is ₹150, GOLD is ₹250, PLATINUM is ₹400 — regardless of which movie, screen or show it belongs to. The total amount for a booking is the sum of the prices of every seat selected.

FR6 — Payment (given). The customer pays using exactly one method per booking — UPI, Card or Cash. If the payment fails, every seat that was tentatively held is released back to AVAILABLE and the booking's status becomes FAILED.

FR7 — Print the ticket. As soon as a booking is confirmed, the system prints exactly one ticket containing the booking id, the movie title, the screen number, the show time, the list of booked seat numbers and the total amount paid.

FR8 — Cancel a booking. A customer can cancel a booking by supplying its booking id. If that id exists and its status is CONFIRMED, every seat held by the booking is set back to AVAILABLE and the booking's status becomes CANCELLED. If the id does not exist, or the booking was never confirmed, the system shows a clear error message and changes nothing.

A.2 Non-Functional Requirements

NFR1 — Modularity. Every class lives in its own file. Changing how a ticket is printed or formatted must never require touching the seat-booking or pricing logic.

NFR2 — Extensibility. Adding a new payment method such as NetBanking must only require writing one new class that follows the existing Payment contract — no existing class should need to be edited.

NFR3 — Robustness / no crashes. An invalid menu choice, a non-existent seat number, or any other malformed input must be caught and reported with a clear message; the program must never terminate abnormally because of bad input.

NFR4 — Consistency. A seat must be shown as BOOKED for a show if and only if there is an actual CONFIRMED booking currently holding that seat for that show — never as a side effect of any other operation.

B. Noun–Verb Analysis

Nouns and candidate classes extracted from the problem statement, with a justification for keeping or rejecting each one.

Noun found	Kept as a class?	Reason
Movie	Yes	Has its own identity and data — title, language, duration — independent of any show.
Seat	Yes	A physical seat with a number and a type; exists whether or not any show is running.
“seat layout”	No	Not a thing in itself — just a view over a Show's seats, so it becomes <code>Show.printLayout()</code> .
Screen	Yes	One auditorium with a fixed number and its own set of physical seats.
Cinema	Yes	The top-level owner that holds every screen, movie and show in the system.
Show	Yes	One screening = a specific movie on a specific screen at a specific time; its seat statuses are unique to it.
Customer	Yes	Has a name and phone number that exist independently of any single booking.
Booking	Yes	Has its own id, status and total amount — a standalone record, not just fields tacked onto a Show.
Ticket	No	Nothing to store — it is only what gets printed for a confirmed booking, so it becomes <code>TicketPrinter.printTicket()</code> .
Payment (UPI / Card / Cash)	Yes, as subclasses	Each method pays in a different way, so each gets its own subclass of an abstract Payment.
Seat number (e.g. “A1”)	No	Just a String attribute that belongs to Seat — not a class of its own.
Amount / Total	No	A derived number — <code>PriceCalculator</code> works it out, <code>Booking</code> only stores the result.
Menu / choice	No	Just what the user types at the console — handled inside the menu loop, not a domain class.

C. The Classes

C.1 Core entity classes

Class	Knows	Does	Must NOT do
Movie	Title, language, duration.	Returns its own details.	Anything about shows, screens or prices.
Seat	Its seat number and seat type.	Returns its own details.	Whether it is booked, or for which show.
Screen	Its screen number and the seats it owns.	Adds and returns its seats.	Movies, shows or bookings.
Cinema	Its name and every screen / movie / show it owns.	Adds new screens / movies / shows; finds shows for a movie.	Booking or payment logic; console I/O.
Show	Its movie, its screen, its start time, and the status of every seat for this screening.	Builds its ShowSeats, looks one up by number, prints the seat layout.	Changing seat status on its own initiative; handling payment.
ShowSeat	The seat it is tracking and whether it is AVAILABLE or BOOKED for this show only.	Flips its own status.	Anything about Booking, Customer or price.
Customer	Name and phone number.	Returns its own details.	Any booking or payment logic.
Booking	Its id, the show, the customer, the seats, the total amount, and its status.	Changes its own status (CONFIRMED / FAILED / CANCELLED).	Calculating price, taking payment, or printing the ticket itself.

C.2 Behaviour / service classes

Class	Knows	Does	Must NOT do
Payment (abstract)	Nothing but the pay() contract.	Declares that every payment type must implement pay(amount).	Actually process any payment itself.
UpiPayment / CardPayment / CashPayment	The one detail it needs (UPI id, card number, or nothing for cash).	Actually attempts the payment and reports success or failure.	Touch Booking or Show directly.
PriceCalculator	Nothing — stateless.	Adds up the price of a list of seats.	Change any seat or booking, or print anything.
TicketPrinter	Nothing — stateless.	Formats and prints a ticket for a confirmed booking.	Change booking, seat or payment state — printing only.
BookingService	Every booking made so far.	Runs the full flow: validate seats → lock them → price → take payment → confirm or roll back → print; also handles cancellation.	Read console input or draw the menu; perform its own payment logic.

Class	Knows	Does	Must NOT do
Cinema / main menu	The Cinema, the BookingService, and the current customer.	Reads console input and drives the menu.	Any pricing, seat-locking or payment logic.

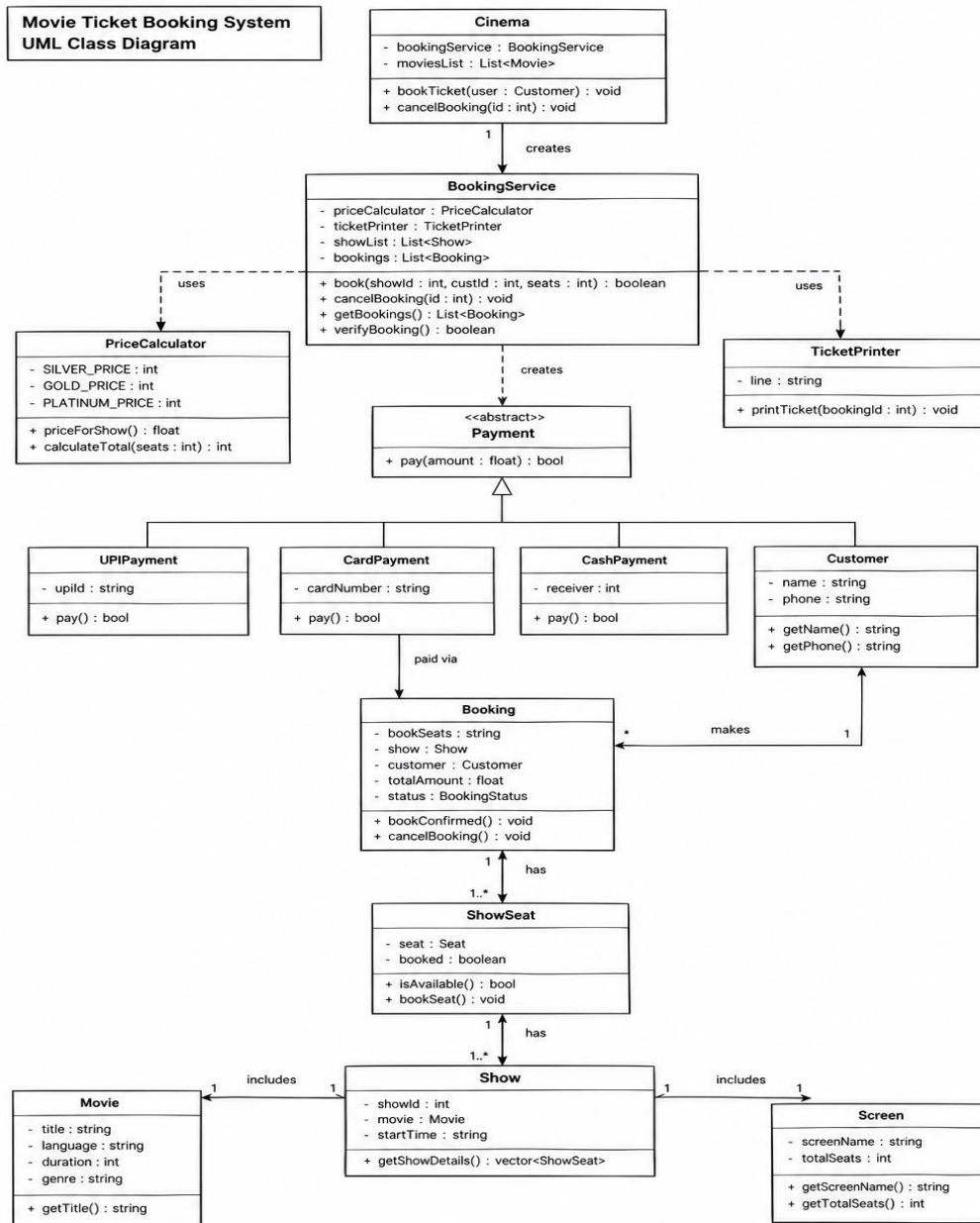
Why ShowSeat and not just Seat? Seat A1 physically exists once on the screen, but its booking status is different for every show — A1 may be booked for the 6 PM show and free for the 9 PM show. Status belongs to the show, not to the physical chair, so it is modelled as its own class, ShowSeat.

D. Relationships

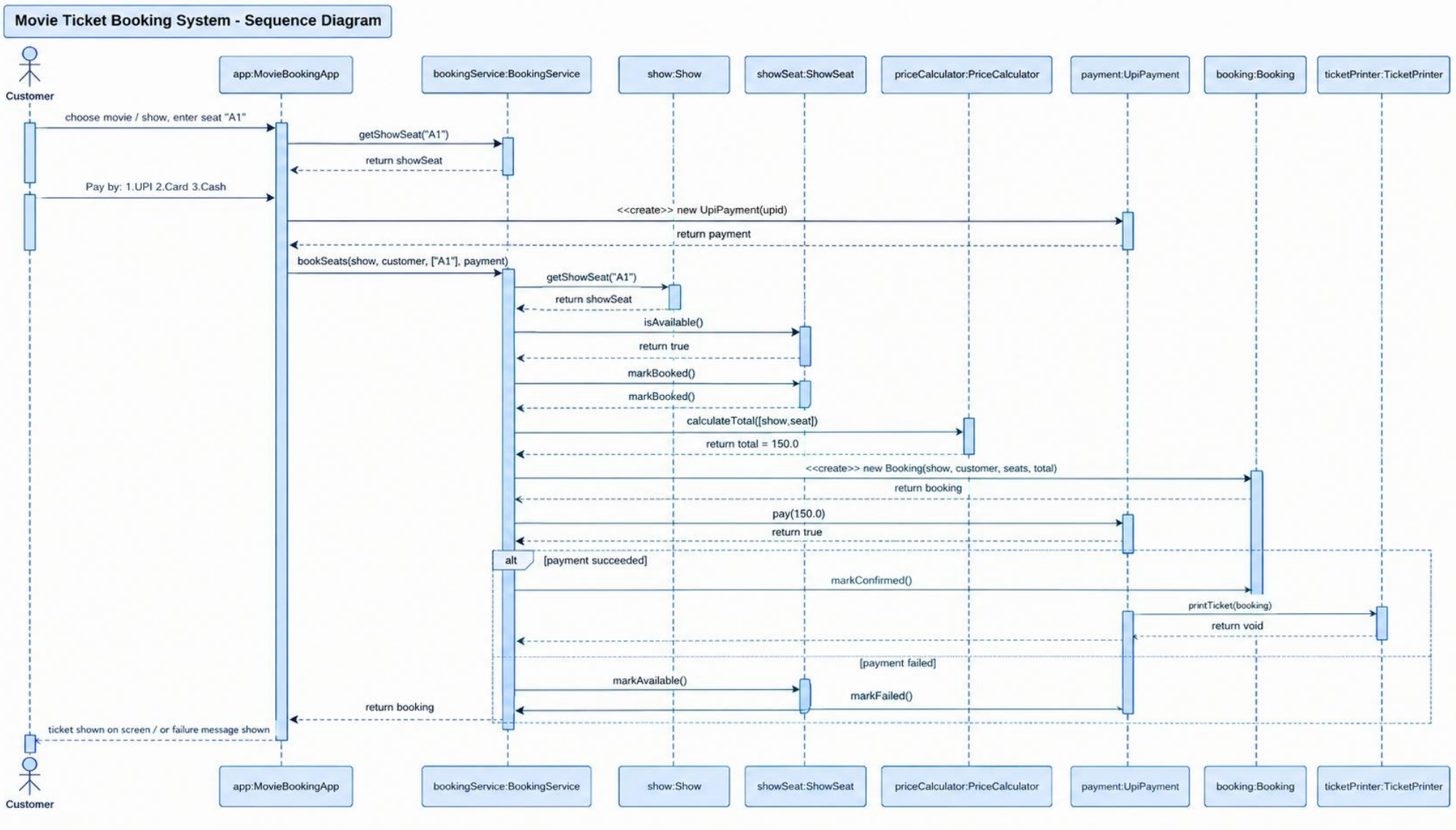
Every pair is decided with the lifetime test: if the whole is destroyed, does the part die with it?

Pair	Relationship	Justification (lifetime test)
Cinema — Screen	Composition	A Cinema creates and owns its Screens. Shut the Cinema down and its Screens go with it — they never exist anywhere else.
Screen — Seat	Composition	Seats belong to exactly one Screen. Remove the Screen and its Seats disappear too; a Seat cannot float on its own.
Show — Movie	Aggregation (given)	A Show only borrows a Movie. Cancel the 6 PM show and “3 Idiots” still exists and still plays at 9 PM.
Show — Screen	Aggregation	A Show borrows a Screen for a time slot. When the show ends the Screen is still there, ready for the next show.
Show — ShowSeat	Composition	A ShowSeat's status only makes sense for one particular show. Delete the Show and its ShowSeats have no reason to exist.
Booking — Customer	Association	A Booking simply references the Customer who made it. The Customer existed before the booking and outlives it — deleting a booking never touches the customer.
Booking — ShowSeat	Aggregation	A Booking refers to seats that already exist. Cancel the booking and the ShowSeats survive — they just return to AVAILABLE.
Booking — Payment	Composition	One Payment attempt belongs to exactly one Booking and has no reason to exist once that booking is gone.
Payment — UpiPayment	Inheritance	UpiPayment is a specific kind of Payment — it inherits the pay() contract and fills it in with UPI-specific behaviour.
BookingService — Booking	Aggregation	BookingService keeps and manages Bookings, but each Booking is its own record that could, in principle, be handed to or stored by something else.

E. Class Diagram



F. Sequence Diagram — Booking One Seat and Paying by UPI



G. SOLID Principles

Principle	How it is applied here
Single Responsibility	Booking never prints a ticket — TicketPrinter owns that job. A change to how a ticket looks touches only TicketPrinter; a change to a booking rule touches only Booking.
Open/Closed	BookingService only ever talks to the abstract Payment type. Adding NetBanking means adding one new subclass — BookingService's code does not change at all.
Liskov Substitution	Any Payment — UpiPayment, CardPayment, CashPayment, or a future NetBankingPayment — can be substituted wherever a Payment is expected. BookingService simply calls pay(total) and trusts every implementation to honour that contract.
Interface Segregation	Payment only demands pay() from its subclasses; it deliberately does not force a refund() method on every payment type, since not every payment method can meaningfully support a refund.
Dependency Inversion	BookingService receives an already-constructed Payment object from its caller instead of instantiating a CardPayment or UpiPayment itself. The booking logic depends only on the Payment abstraction, never on a concrete payment class.

One thing I deliberately did NOT do

I did not add any persistent storage — all bookings live only in memory (a Map inside BookingService) for as long as the program runs. This was left out on purpose to keep the console demo focused on the OOP design the assignment asks for, rather than on file or database handling. If real storage were needed later, it would only mean changing what is inside BookingService — no other class would need to change, which is exactly the Open/Closed and Dependency Inversion principles doing their job. For the same reason, I did not add a refund() method to Payment: cash and UPI refunds do not behave the same way, and forcing every payment type to implement a method nobody asked for would violate Interface Segregation.

Demo Output:

```
aman@amansg MINGW64 ~/OneDrive/Desktop/Ticket Booking System/src/app (main)
$ ./main
```

```
===== MOVIE TICKET BOOKING =====
```

1. List movies
2. List shows
3. Show seats
4. Book seats
5. Cancel booking
6. Exit

Choice: 2

--- Movies Currently Playing ---

1. Hanuman Ansh | Hindi | 159 min
2. Godfather part 2 | English | 218 min

Movie number: 1

Show 1 | Screen 1 | 06:00 PM

Show 2 | Screen 1 | 09:00 PM

```
===== MOVIE TICKET BOOKING =====
```

1. List movies
2. List shows
3. Show seats
4. Book seats
5. Cancel booking
6. Exit

Choice: 3

Show ID: 1

```
===== SEAT LAYOUT =====
SCREEN
```

Row 1 [1 A] [2 A] [3 A] [4 A]

Row 2 [5 A] [6 A] [7 A] [8 A]

Row 3 [9 A] [10 A] [11 A] [12 A]

=====

A = Available X = Booked

Silver: 1-4 | Gold: 5-8 | Platinum: 9-12

```
===== MOVIE TICKET BOOKING =====
```

1. List movies
2. List shows
3. Show seats
4. Book seats
5. Cancel booking
6. Exit

Choice: 4

Show ID: 1

```
===== SEAT LAYOUT =====
SCREEN
```

Row 1 [1 A] [2 A] [3 A] [4 A]

Row 2 [5 A] [6 A] [7 A] [8 A]

Row 3 [9 A] [10 A] [11 A] [12 A]

=====

A = Available X = Booked

Silver: 1-4 | Gold: 5-8 | Platinum: 9-12

Number of seats: 1

Enter seat numbers: 2

Payment: 1.UPI 2.Card 3.Cash : 1

Total amount: Rs.150

Processing UPI payment Rs.150...

```
===== TICKET =====
```

Booking ID : 1001

Movie : Hanuman Ansh

Screen : 1

Time : 06:00 PM

Seats : 2

Total : Rs.150

=====

```
===== MOVIE TICKET BOOKING =====
```

1. List movies
2. List shows
3. Show seats
4. Book seats
5. Cancel booking
6. Exit

Choice: 5

Booking ID: 1

Booking not found.

```
===== MOVIE TICKET BOOKING =====
```

1. List movies
2. List shows
3. Show seats
4. Book seats
5. Cancel booking
6. Exit

Choice: 5

Booking ID: 1001

Booking 1001 cancelled. Seats are AVAILABLE again.

```
===== MOVIE TICKET BOOKING =====
```

1. List movies
2. List shows
3. Show seats
4. Book seats
5. Cancel booking
6. Exit

Choice: 6

Thank you!